NEXNOTE - COMPLETE DEVOPS IMPLEMENTATION REPORT

Prepared By: Anjali Kumari
Date: 24 May
Repository: https://github.com/Anjali11s/NexNoteDevOps

Table of Contents

1. PROJECT OVERVIEW

1.1 Project Description

NexNote is a full-stack notes application that demonstrates complete DevOps automation from code commit to production deployment with zero downtime and real-time monitoring.

1.2 Application Components

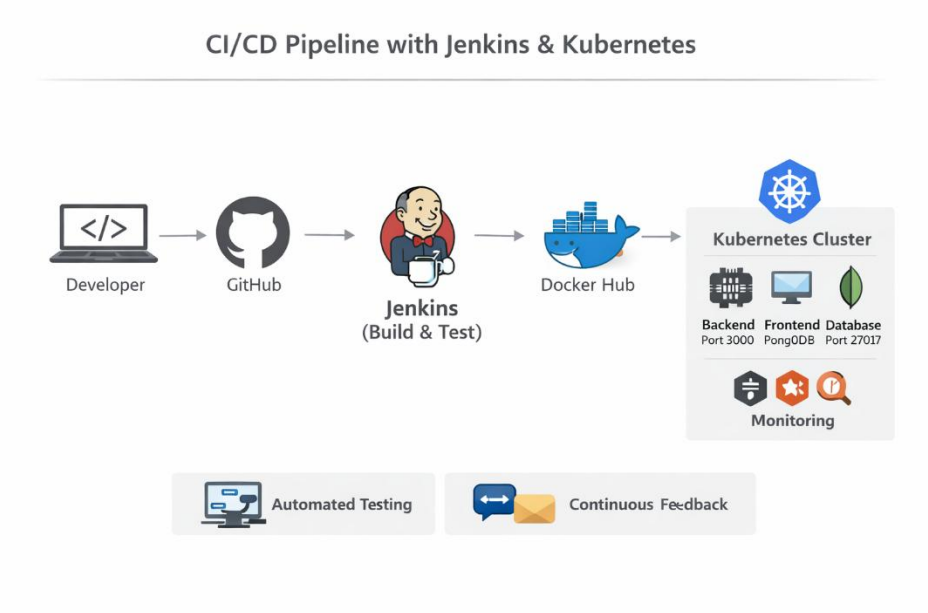
Component	Technology	Description
Frontend	React.js, Vite, Tailwind CSS	User interface for creating/managing notes
Backend	Node.js, Express.js	REST API for authentication and CRUD operations

Component	Technology	Description
Database	MongoDB	Persistent storage for users and notes

1.3 DevOps Goals Achieved

Goal	Implementation
Continuous Integration	GitHub Actions runs tests on every push
Continuous Deployment	Jenkins automates Kubernetes deployment
Containerization	Docker images for frontend & backend
Orchestration	Kubernetes manages scaling & self-healing
Monitoring	Prometheus collects metrics; Grafana visualizes
Zero Downtime	Rolling updates in Kubernetes
Alerting	Alertmanager sends notifications on failures

2. ARCHITECTURE DESIGN

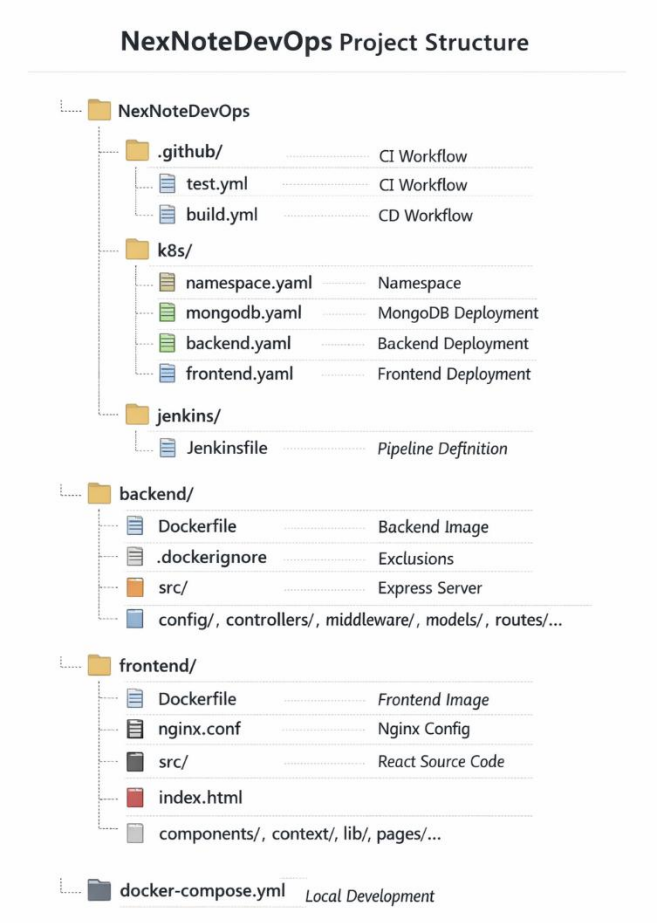


3. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	**React.js, Vite, Tailwind CSS**	User interface
Backend	**Node.js, Express.js**	REST API
Database	**MongoDB**	Data persistence
Container	**Docker**	Application packaging
Orchestration	**Kubernetes (K3s)**	Container management
CI	**GitHub Actions**	Testing & building
CD	**Jenkins**	Deployment automation
Registry	**Docker Hub**	Image storage
Monitoring	**Prometheus**	Metrics collection

Layer	Technology	Purpose
Visualization	**Grafana**	Dashboards
Alerting	**Alertmanager**	Notification system
Cloud	**AWS EC2**	Infrastructure

4. PROJECT STRUCTURE



5. DOCKER CONTAINERIZATION

5.1 Backend Dockerfile

- Multi-stage build for smaller production image

FROM node:20-alpine AS builder

```
WORKDIR /app

COPY package*.json ./

RUN npm ci --only=production


FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/node_modules ./node_modules

COPY . .


EXPOSE 5001


HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \

  CMD node -e "require('http').get('http://localhost:5001/api/health', (r) => {process.exit(r.statusCode
=== 200 ? 0 : 1)})"


CMD ["node", "server.js"]
```

5.2 Frontend Dockerfile (Multi-stage)

dockerfile

```
# Stage 1: Build React app

FROM node:20-alpine AS build


WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build


# Stage 2: Serve with nginx

FROM nginx:alpine


COPY --from=build /app/dist /usr/share/nginx/html
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

5.3 Nginx Configuration (Reverse Proxy)

```
nginx
server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html;

    # React Router support
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API reverse proxy to backend
    location /api {
        proxy_pass http://backend-service.nexnote:5001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

5.4 Benefits of Multi-stage Builds

Feature	Benefit
Alpine Linux	Small image size (~5MB vs 100MB)
Multi-stage	Only production dependencies in final image
Health check	Docker can monitor container health
Layer caching	Faster builds on subsequent runs

6. GITHUB ACTIONS CI PIPELINE

6.1 Test Workflow (test.yml)

```
name: Test Application

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test-backend:
    runs-on: ubuntu-latest
    services:
      mongodb:
        image: mongo:6
        ports:
          - 27017:27017
    steps:
      - uses: actions/checkout@v4
```

```
    - uses: actions/setup-node@v4
      with:
        node-version: '20'
    - run: cd backend && npm ci
    - run: cd backend && npm test
      env:
        MONGO_URI: mongodb://localhost:27017/test
        JWT_SECRET: test_secret

  test-frontend:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: '20'
    - run: cd frontend && npm ci
    - run: cd frontend && npm run build

  test-docker-build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3
    - name: Test backend Docker build
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        push: false
        load: true
```

```
      tags: nexnote-backend:test
    - name: Test frontend Docker build
      uses: docker/build-push-action@v5
      with:
        context: ./frontend
        push: false
        load: true
        tags: nexnote-frontend:test
```

6.2 Build Workflow (build.yml)

yaml

```
name: Build and Push Images

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_TOKEN }}

      - name: Get commit SHA
        id: vars
        run: echo "sha_short=$(git rev-parse --short HEAD)" >> $GITHUB_OUTPUT
```

```
    - name: Build and push Backend
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        push: true
        tags: |
          ${{ secrets.DOCKER_USERNAME }}/nexnote-backend:latest
          ${{ secrets.DOCKER_USERNAME }}/nexnote-backend:${{ steps.vars.outputs.sha_short }}

    - name: Build and push Frontend
      uses: docker/build-push-action@v5
      with:
        context: ./frontend
        push: true
        tags: |
          ${{ secrets.DOCKER_USERNAME }}/nexnote-frontend:latest
          ${{ secrets.DOCKER_USERNAME }}/nexnote-frontend:${{ steps.vars.outputs.sha_short }}

    - name: Update image tags in k8s manifests
      run: |
        sed -i "s|nexnote-backend:.*|nexnote-backend:${{ steps.vars.outputs.sha_short }}|g" k8s/backend.yaml
        sed -i "s|nexnote-frontend:.*|nexnote-frontend:${{ steps.vars.outputs.sha_short }}|g" k8s/frontend.yaml

    - name: Commit updated manifests
      run: |
        git config user.name "GitHub Actions Bot"
        git config user.email "actions@github.com"
        git add k8s/*.yaml
        git commit -m "Update image tags to ${{ steps.vars.outputs.sha_short }}" || echo "No changes"
```

```
        git push

    - name: Trigger Jenkins job
      run: |
        curl -X POST "${{ secrets.JENKINS_URL }}/job/NexNote-Deploy/buildWithParameters" \
          --user ${{ secrets.JENKINS_USER }}:${{ secrets.JENKINS_TOKEN }} \
          --data-urlencode "IMAGE_TAG=${{ steps.vars.outputs.sha_short }}"
```

6.3 GitHub Secrets Configuration

Secret Name	Value	Purpose
DOCKER_USERNAME	Docker Hub username	Image registry authentication
DOCKER_TOKEN	Docker Hub access token	Push images to Docker Hub
JENKINS_URL	http://EC2_PUBLIC_IP:8080	Jenkins server endpoint
JENKINS_USER	Jenkins username	API authentication
JENKINS_TOKEN	Jenkins API token	API authentication

7. JENKINS CD PIPELINE

7.1 Jenkinsfile

groovy

```
pipeline {
    agent any

    parameters {
        string(name: 'IMAGE_TAG', defaultValue: 'latest', description: 'Docker image tag to deploy')
    }

    stages {
```

```
stage('Deploy Backend') {
    steps {
        script {
            sh """
                kubectl set image deployment/backend \
                    backend=singhanjali1/nexnote-backend:${params.IMAGE_TAG} \
                    -n nexnote
            """
        }
    }
}

stage('Deploy Frontend') {
    steps {
        script {
            sh """
                kubectl set image deployment/frontend \
                    frontend=singhanjali1/nexnote-frontend:${params.IMAGE_TAG} \
                    -n nexnote
            """
        }
    }
}

stage('Verify Deployment') {
    steps {
        script {
            sh """
                kubectl rollout status deployment/backend -n nexnote --timeout=3m
                kubectl rollout status deployment/frontend -n nexnote --timeout=3m
            """
```

```
            }
        }
    }
}

    post {
        success {
            echo '✅ Deployment successful!'
        }
        failure {
            echo '❌ Deployment failed!'
        }
    }
}
```

7.2 Jenkins Setup

```
# Run Jenkins container
docker run -d \
  --name jenkins \
  --restart always \
  -p 8080:8080 \
  -p 50000:50000 \
  -v ~/jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts-jdk17

# Get initial password
docker logs jenkins 2>&1 | grep -A 10 "Please use the following password"
```

7.3 Jenkins Job Configuration

Setting	Value
Job Name	NexNote-Deploy
Type	Pipeline
Parameterized	✓ Yes
Parameter Name	IMAGE_TAG
Default Value	latest
Build Trigger	Trigger builds remotely
Token	nexnote-token-123
SCM	Git
Repository URL	https://github.com/Anjali11s/NexNoteDevOps.git
Branch	*/main
Script Path	jenkins/Jenkinsfile

7.4 Kubectl Installation in Jenkins Container

```
# Install kubectl inside Jenkins container
docker exec -it jenkins bash
apt-get update
apt-get install -y curl
curl -LO "https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl"
chmod +x kubectl
mv kubectl /usr/local/bin/
exit
```

```
# Configure kubeconfig
sudo cat /etc/rancher/k3s/k3s.yaml | docker exec -i jenkins tee /var/jenkins_home/.kube/config
docker exec -it jenkins chmod 600 /var/jenkins_home/.kube/config
docker restart jenkins
```

8. KUBERNETES ORCHESTRATION (K3s)

8.1 K3s Installation on EC2

```
# Install K3s
curl -sfL https://get.k3s.io | sh -

# Configure kubeconfig
mkdir -p ~/.kube
sudo cat /etc/rancher/k3s/k3s.yaml > ~/.kube/config
sudo chown ubuntu:ubuntu ~/.kube/config
chmod 600 ~/.kube/config

# Verify
kubectl get nodes
```

8.2 Docker Hub Secret for Image Pull

```
bash
kubectl create secret docker-registry dockerhub-secret \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=singhanjali1 \
  --docker-password=dckr_pat_xxxxxxxxxxx \
  --docker-email=email@example.com \
  -n nexnote
```

8.3 Kubernetes Manifests

Namespace (namespace.yaml)

```
yaml
apiVersion: v1
```

```
kind: Namespace
metadata:
  name: nexnote
```

MongoDB (mongodb.yaml)

```
yaml
apiVersion: v1
kind: Secret
metadata:
  name: mongodb-secret
  namespace: nexnote
type: Opaque
data:
  username: YWRtaW4=
  password: c2VjcmV0MTIz
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-pvc
  namespace: nexnote
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb
```

```
  namespace: nexnote
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
      - name: mongodb
        image: mongo:6
        ports:
        - containerPort: 27017
        env:
        - name: MONGO_INITDB_ROOT_USERNAME
          valueFrom:
            secretKeyRef:
              name: mongodb-secret
              key: username
        - name: MONGO_INITDB_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mongodb-secret
              key: password
        volumeMounts:
        - name: mongodb-storage
          mountPath: /data/db
      volumes:
```

```
      - name: mongodb-storage
        persistentVolumeClaim:
          claimName: mongodb-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb-service
  namespace: nexnote
spec:
  selector:
    app: mongodb
  ports:
  - port: 27017
    targetPort: 27017
```

Backend (backend.yaml)

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: nexnote
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
```

```
spec:
  imagePullSecrets:
  - name: dockerhub-secret
  containers:
  - name: backend
    image: singhanjali1/nexnote-backend:latest
    ports:
    - containerPort: 5001
    env:
    - name: NODE_ENV
      value: "production"
    - name: PORT
      value: "5001"
    - name: JWT_SECRET
      value: "mysecretkey12345"
    - name: MONGO_URI
      value: "mongodb://admin:secret123@mongodb-service:27017/nexnote?authSource=admin"
    resources:
      requests:
        memory: "256Mi"
        cpu: "250m"
      limits:
        memory: "512Mi"
        cpu: "500m"
    livenessProbe:
      httpGet:
        path: /api/health
        port: 5001
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:
```

```
        httpGet:
          path: /api/health
          port: 5001
        initialDelaySeconds: 5
        periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: nexnote
spec:
  selector:
    app: backend
  ports:
  - port: 5001
    targetPort: 5001
  type: NodePort
```

Frontend (frontend.yaml)

```
yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: nexnote
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
```

```
    metadata:
      labels:
        app: frontend
    spec:
      imagePullSecrets:
      - name: dockerhub-secret
      containers:
      - name: frontend
        image: singhanjali1/nexnote-frontend:latest
        ports:
        - containerPort: 80
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "200m"
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: nexnote
spec:
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 80
  type: NodePort
```

8.4 Deployment Commands

bash

Apply all manifests

kubectl apply -f k8s/namespace.yaml

kubectl apply -f k8s/mongodb.yaml

kubectl apply -f k8s/backend.yaml

kubectl apply -f k8s/frontend.yaml

Check status

kubectl get pods -n nexnote -w

kubectl get svc -n nexnote

8.5 Kubernetes Features Demonstrated

Feature	How it works	Benefit
Self-Healing	Kubernetes auto-restarts failed pods	High availability
Rolling Update	Gradual pod replacement	Zero downtime
Load Balancing	Service distributes traffic	Even load distribution
Health Checks	Liveness & readiness probes	Automatic failure detection
Persistent Storage	PVC for MongoDB	Data persists across restarts

9. MONITORING WITH PROMETHEUS & GRAFANA

9.1 Install kube-prometheus-stack via Helm

Install Helm

sudo snap install helm --classic

Add repository

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

```
helm repo update

# Create namespace
kubectl create namespace monitoring

# Install stack
helm install prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword=admin123 \
  --set prometheus.prometheusSpec.retention=7d
```

9.2 Expose Services

```
# Expose Grafana
kubectl patch svc prometheus-stack-grafana -n monitoring -p '{"spec": {"type": "NodePort"}}'
kubectl get svc prometheus-stack-grafana -n monitoring
# Output: 80:31729/TCP

# Expose Prometheus
kubectl patch svc prometheus-stack-kube-prom-prometheus -n monitoring -p '{"spec": {"type": "NodePort"}}'
kubectl get svc prometheus-stack-kube-prom-prometheus -n monitoring
# Output: 9090:31691/TCP
```

9.3 Add Prometheus Data Source in Grafana

text

Grafana → Connections → Data sources → Add Prometheus

URL: http://prometheus-stack-kube-prom-prometheus.monitoring:9090

Save & Test → Green check

9.4 Six Custom Dashboards for NexNote

Panel	PromQL Query	Visualization
Deployment Replicas	kube_deployment_status_replicas{namespace="nexnote"}	Table

Panel	PromQL Query	Visualization
Pod Restarts	kube_pod_container_status_restarts_total{namespace="nexnote"}	Table
Running Pods Count	count(kube_pod_status_phase{namespace="nexnote", phase="Running"})	Stat
CPU Usage	sum(rate(container_cpu_usage_seconds_total{namespace="nexnote"}[5m]))	Time series
Memory Usage	sum(container_memory_working_set_bytes{namespace="nexnote"}) / 1024 / 1024	Stat
Pod Status Ready	kube_pod_status_ready{namespace="nexnote", condition="true"}	Table

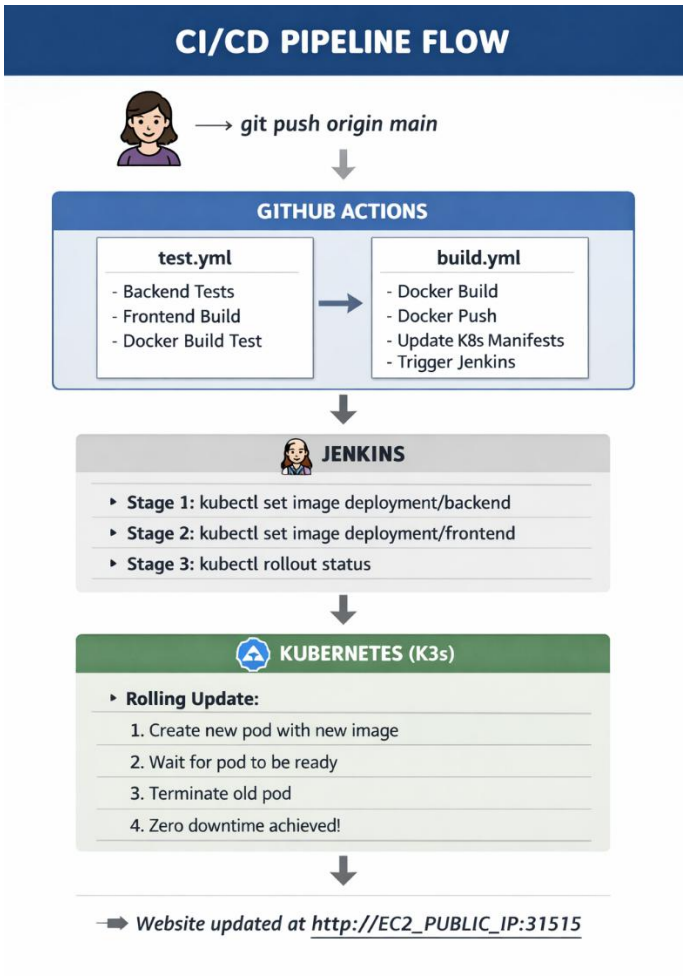
9.5 Access URLs

Service	URL
Frontend App	http://EC2_PUBLIC_IP:31515
Jenkins	http://EC2_PUBLIC_IP:8080
Grafana	http://EC2_PUBLIC_IP:31729
Prometheus	http://EC2_PUBLIC_IP:31691
Alertmanager	http://EC2_PUBLIC_IP:30748

Grafana Credentials: admin / admin123

10. COMPLETE WORKFLOW

10.1 End-to-End Flow



10.2 Role of Each Tool

Tool	Role
GitHub Actions	Builds Docker images and pushes to Docker Hub
Jenkins	Runs kubectl commands to update Kubernetes deployments
Kubernetes	Performs rolling update with zero downtime

10.3 Self-Healing Demonstration

Terminal 1 - Watch pods

kubectl get pods -n nexnote -w

Terminal 2 - Delete a pod

kubectl delete pod <pod-name> -n nexnote

Kubernetes automatically recreates the pod!

11. EC2 SETUP & RESTART PROCEDURE

11.1 Initial EC2 Setup

Update system

sudo apt update && sudo apt upgrade -y

Install Docker

curl -fsSL https://get.docker.com | sudo sh

sudo usermod -aG docker ubuntu

newgrp docker

Install K3s

curl -sfL https://get.k3s.io | sh -

Clone repository

git clone https://github.com/Anjali11s/NexNoteDevOps.git

cd NexNoteDevOps

Deploy applications

kubectl create namespace nexnote

kubectl apply -f k8s/mongodb.yaml

sleep 30

kubectl apply -f k8s/backend.yaml

kubectl apply -f k8s/frontend.yaml

Install monitoring

```
sudo snap install helm --classic

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

helm repo update

kubectl create namespace monitoring

helm install prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword=admin123


# Expose monitoring services

kubectl patch svc prometheus-stack-grafana -n monitoring -p '{"spec": {"type": "NodePort"}}'

kubectl patch svc prometheus-stack-kube-prom-prometheus -n monitoring -p '{"spec": {"type": "NodePort"}}'
```

11.2 EC2 Restart Procedure

```
# 1. Start EC2 instance from AWS Console

# 2. Get new public IP (e.g., 16.16.205.6)


# 3. SSH into EC2 (same SSH key works)

ssh -i "nexnote-key.pem" ubuntu@16.16.205.6


# 4. Start Jenkins container

docker start jenkins


# 5. Update GitHub secret JENKINS_URL

# New value: http://16.16.205.6:8080


# 6. Verify everything works

kubectl get pods -n nexnote

kubectl get svc -n nexnote

curl http://16.16.205.6:31515


# 7. Access applications (ex)
```

Frontend: http://16.16.205.6:31515

Jenkins: http://16.16.205.6:8080

Grafana: http://16.16.205.6:31729

Prometheus: http://16.16.205.6:31691

Alertmanager: http://16.16.205.6:30748

11.3 What Changes vs What Doesn't

Item	Changes?	Action Required
Public IP	Yes	Update browser URLs & GitHub secret
Private IP	Yes	No action (K3s auto-updates)
SSH Key	No	Same key works
Data	No	Safe on EBS volume
Pods	Restart automatically	No action

12. TROUBLESHOOTING GUIDE

12.1 Common Issues and Solutions

Issue	Diagnosis	Solution
ImagePullBackOff	kubectl describe pod	Check dockerhub-secret
CrashLoopBackOff	kubectl logs <pod>	Check environment variables
Jenkins 403 Error	CSRF protection	Use crumb or disable CSRF
kubectl not found	which kubectl	Install kubectl in Jenkins container
CORS error	Browser console	Add origin to allowedOrigins

Issue	Diagnosis	Solution
Connection refused	curl localhost:5001	Check if service is running

12.2 Debugging Commands

Check pod logs

kubectl logs -n nexnote <pod-name>

Check pod details

kubectl describe pod -n nexnote <pod-name>

Check service endpoints

kubectl get endpoints -n nexnote

Check Jenkins logs

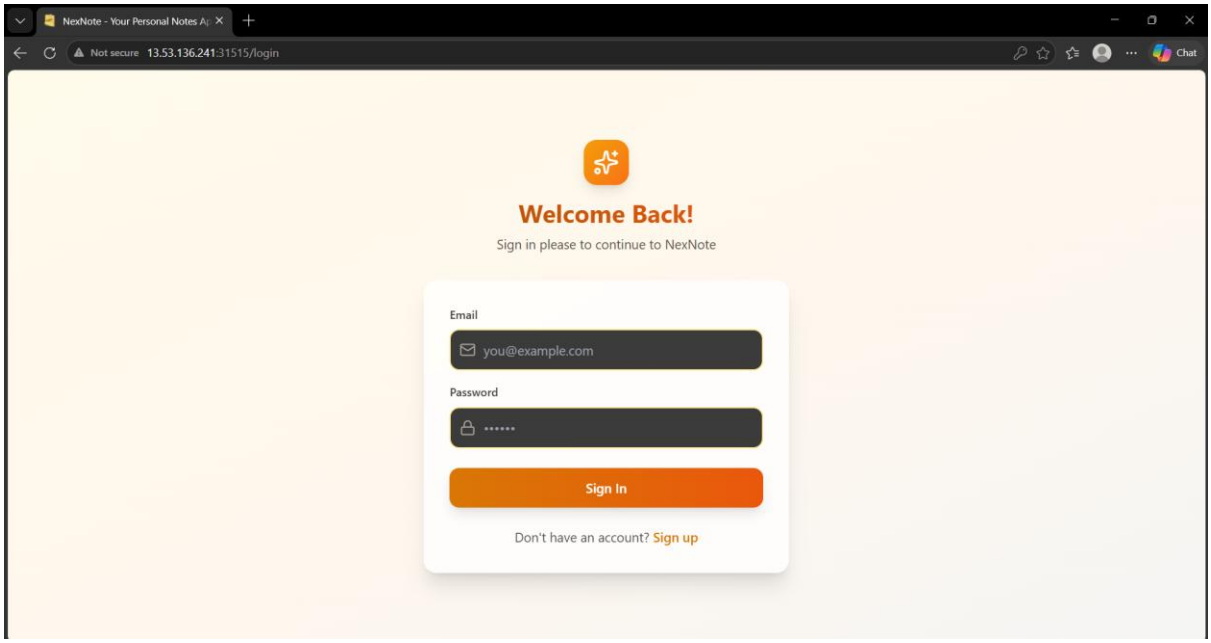
docker logs jenkins --tail 50

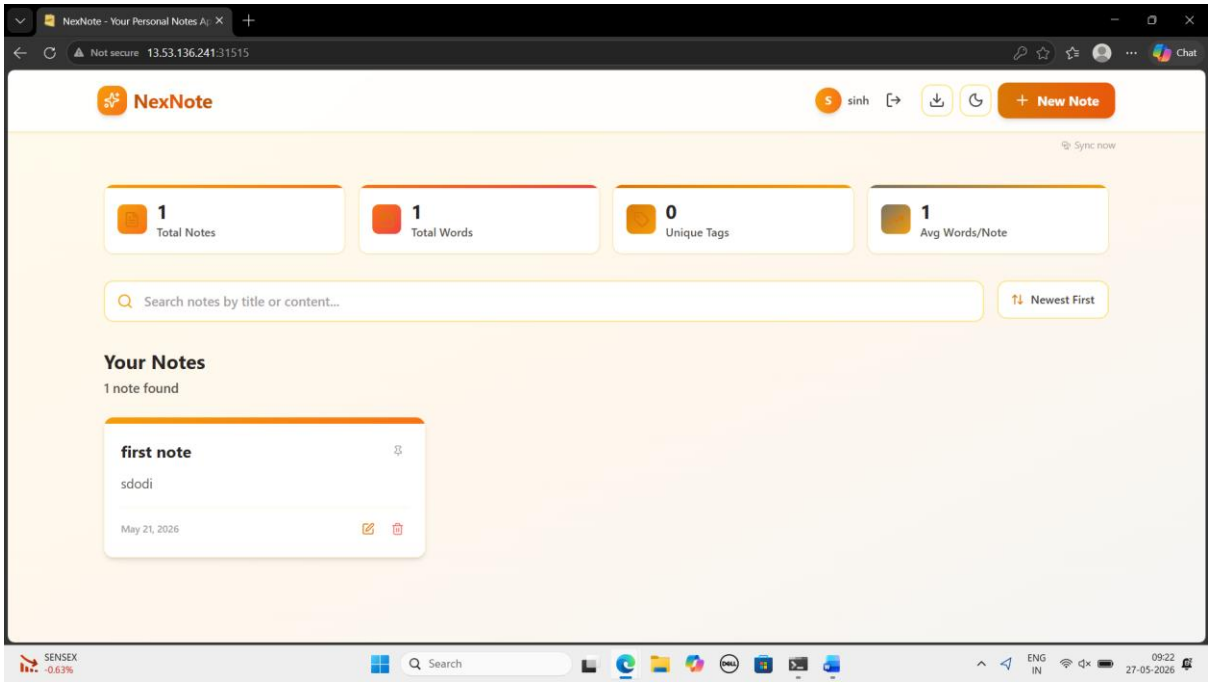
Check K3s status

sudo systemctl status k3s

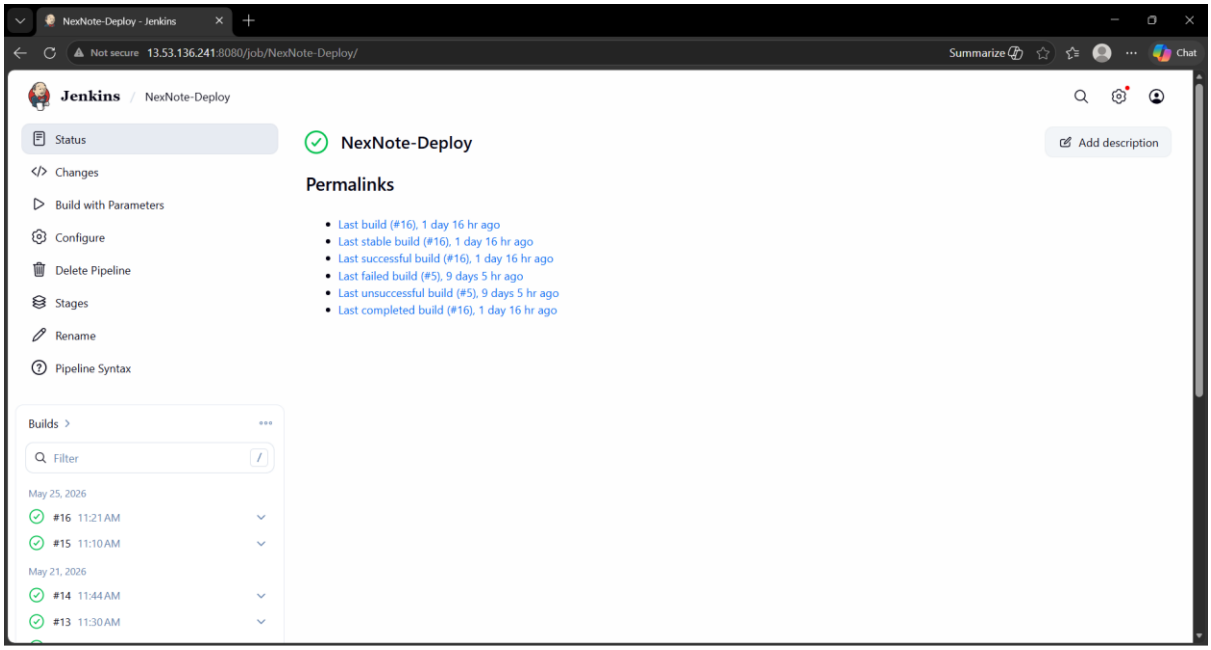
13. Snapshot of Working Programme

Deployment on k8s:

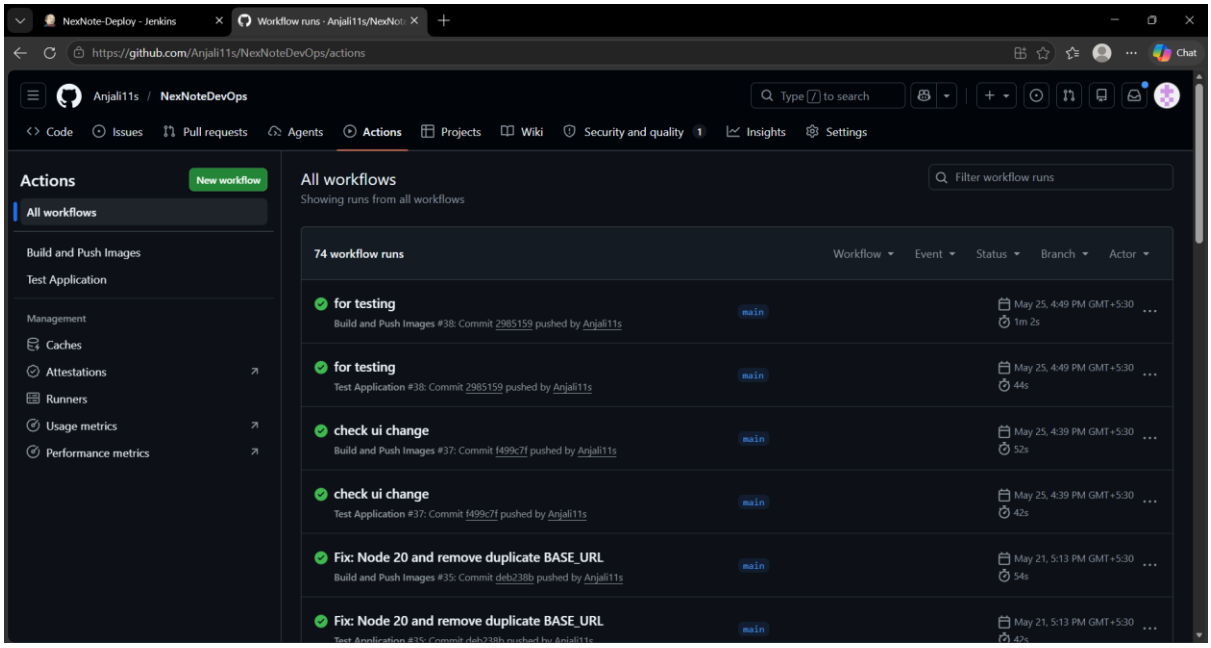




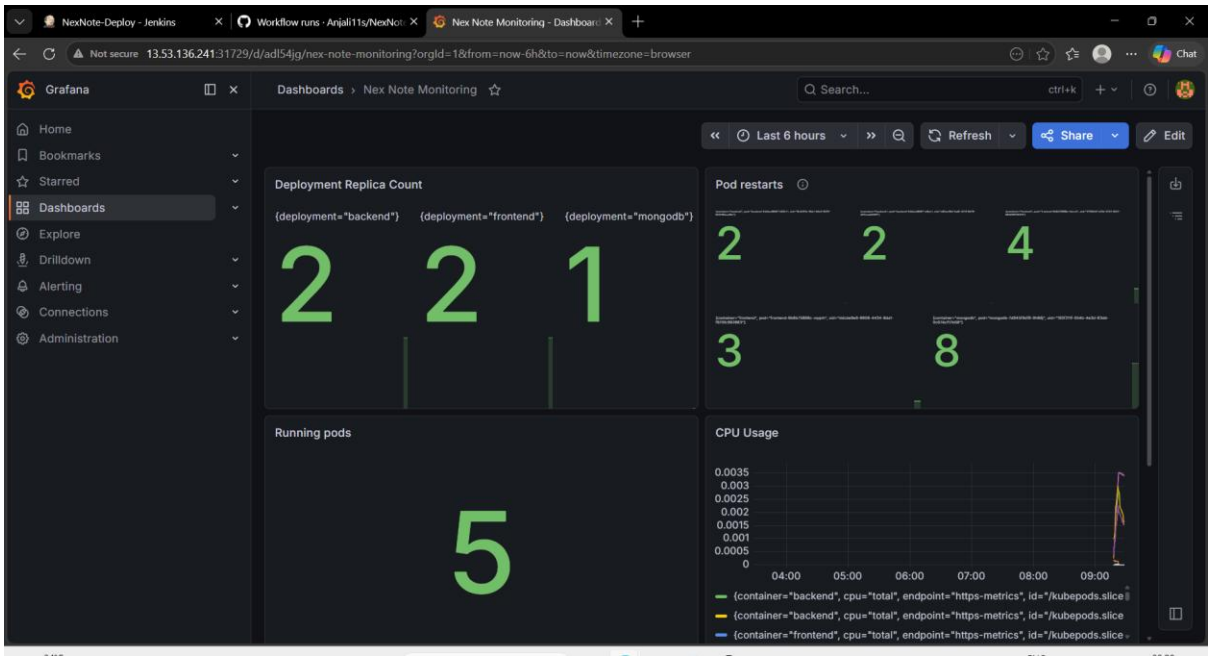
On Jenkins:

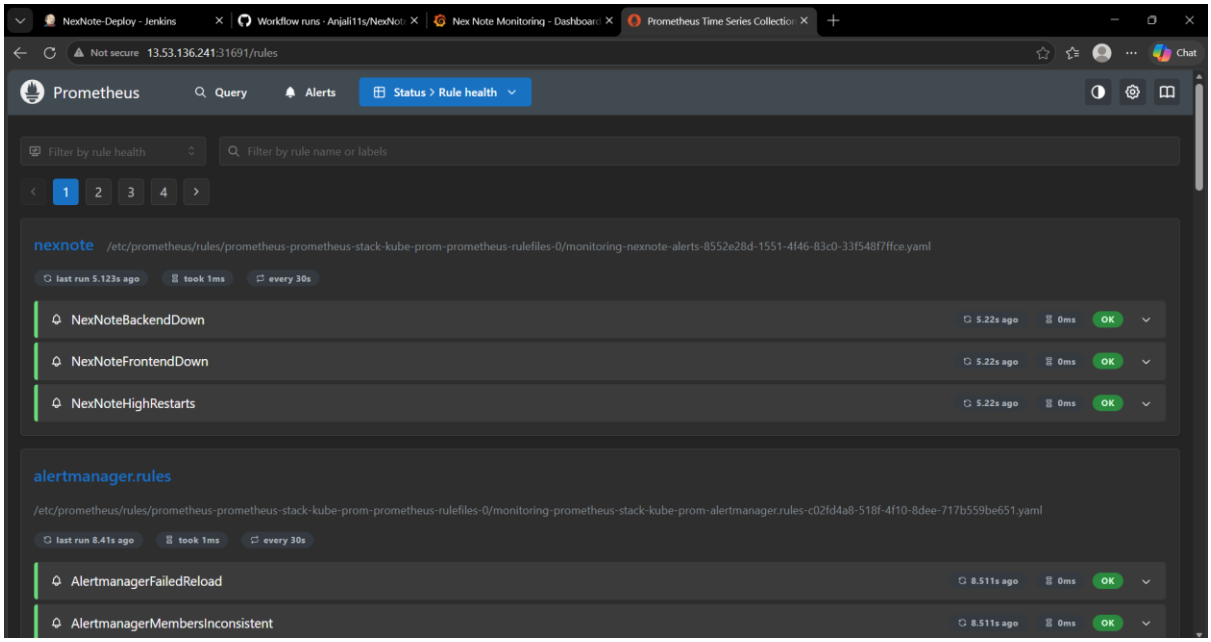


GitHub Action:

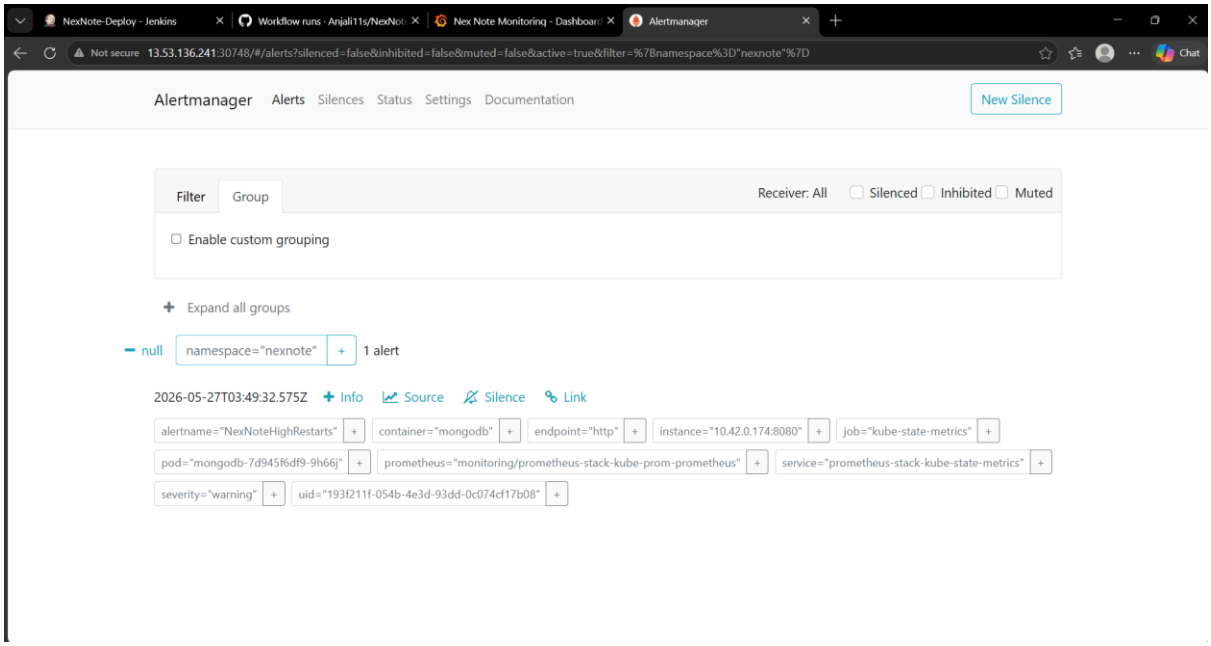


Prometheus & Grafana:

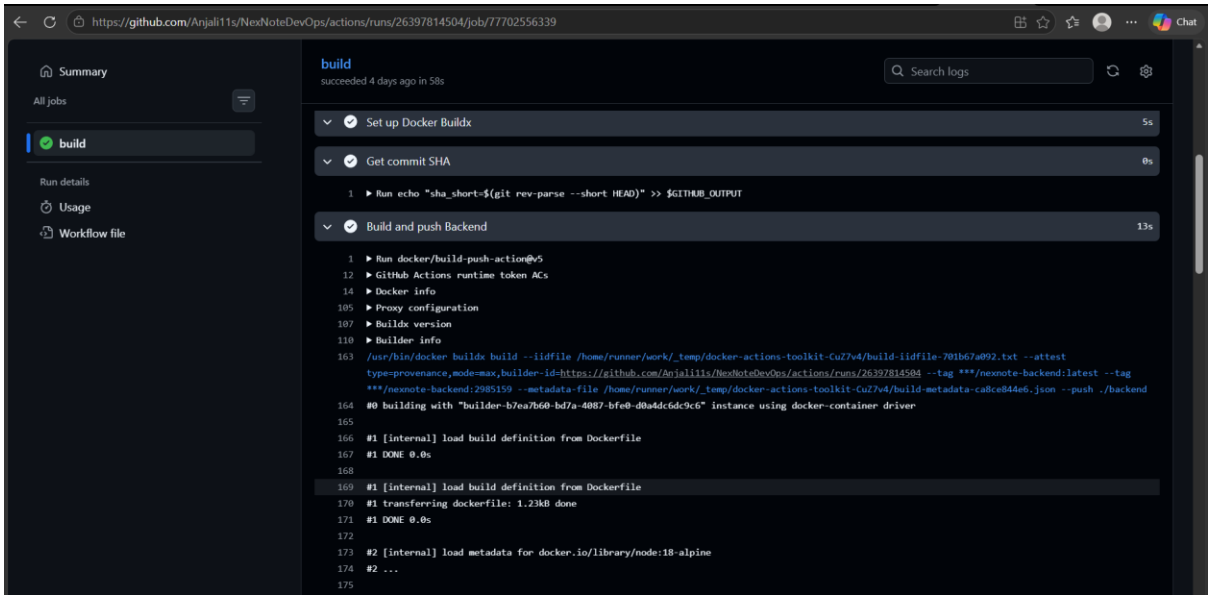




AlertManager:

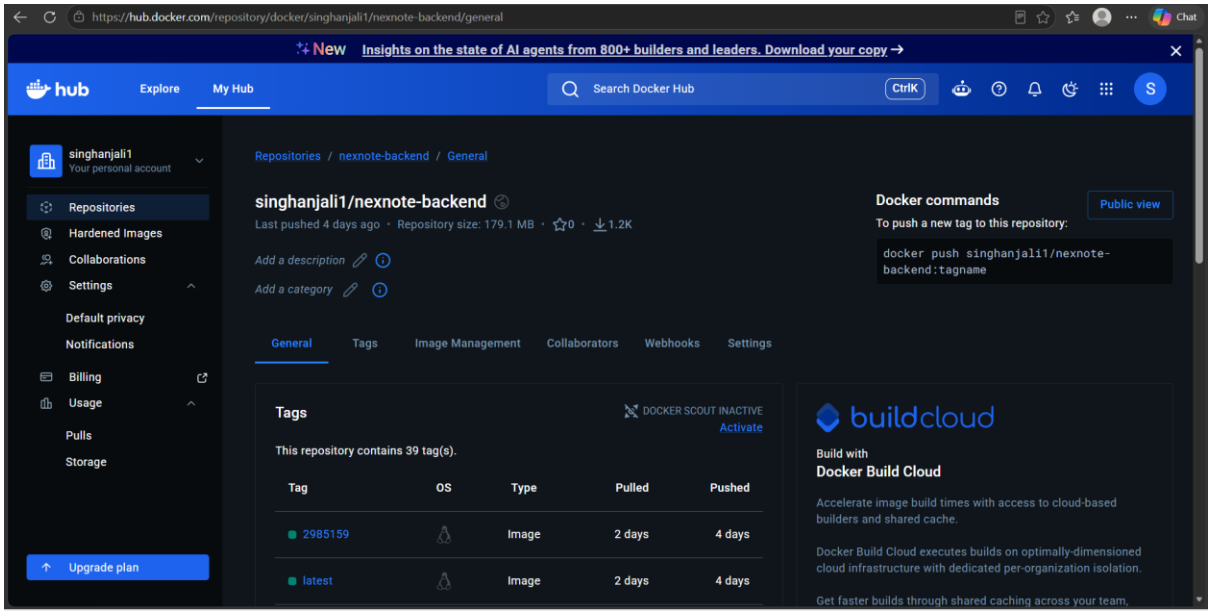


Docker



Here, we can see the image is getting pushed to DockerHub on nexnote-backend:latest

And In DockerHub,



14. CONCLUSION

14.1 Achievements

Goal	Status
Complete CI/CD Pipeline	Fully automated from push to deploy
Zero Downtime Deployments	Rolling updates in Kubernetes
Containerization	Docker images for all services
Orchestration	Kubernetes managing all pods
Monitoring	Prometheus + Grafana with 6 custom panels
Alerting	Alertmanager with custom rules
Self-Healing	Kubernetes auto-restarts failed pods

14.2 Future Scope

Feature	Description
ArgoCD	GitOps for automated sync
Slack Alerts	Real-time notifications on failures
Load Testing	Performance testing with k6
SSL/TLS	Ingress with HTTPS
Horizontal Pod Autoscaling	Auto-scale based on CPU/memory

QUICK REFERENCE

Ports Summary

Port	Service
31515	Frontend App
8080	Jenkins
31729	Grafana
31691	Prometheus
30748	Alertmanager

Daily Commands

```
bash
# Check all pods
kubectl get pods -n nexnote

# Check all services
kubectl get svc -n nexnote

# Restart Jenkins
docker start jenkins

# Scale deployment
kubectl scale deployment backend --replicas=3 -n nexnote

# View logs
kubectl logs -n nexnote <pod-name>
```